ICT Engineering for Smart Societies

Course: Advanced Machine Learning for Image Processing

Laboratory 2 Report

Academic Year 2025/2026

Students:

Lorenzo Cavallaro - s346742
Emanuele Cumino - s361790

Contents

1 Point Cloud Classification with Graph Neural Networks **3**
1.1 Introduction 4
1.2 Methodology 4
1.2.1 Dataset 4
1.2.2 Graph Construction 5
1.2.3 Graph Convolutional Layer 5
1.2.4 Network Architecture 6
1.2.5 Training Protocol 6
1.2.6 Evaluation Metrics 7
1.3 Experiments & Results 7
1.3.1 Test Set Classification Accuracy 7
1.3.2 Confusion Matrix Analysis 8
1.3.3 Prediction Confidence Distribution 8
1.3.4 Failure Case Analysis 9
1.4 Conclusion 10

Abstract

1 Point Cloud Classification with Graph Neural Networks

Abstract

This report presents the implementation and evaluation of a Graph Neural Network (GNN) for binary point cloud classification (chair vs. table) on the ModelNet10 dataset subset. We implement a two-layer GCN architecture from scratch without external graph libraries, utilizing k-nearest neighbor graph construction and symmetric normalized graph convolutions. The model is trained on 1281 normalized point clouds with 2048 points each and evaluated on a balanced test set of 200 samples. Our approach achieves 99% classification accuracy with high confidence on correct predictions and clear separation between confident and uncertain predictions. This work validates that direct learning from unstructured 3D point data via graph neural networks provides an effective alternative to voxel-based methods, eliminating quantization artifacts while preserving spatial relationships.

1.1 Introduction

In this exercise, we implemented, trained and evaluated a Graph Neural Network (GNN) for point cloud classification on a subset of the ModelNet10 dataset. The dataset comprises 1281 training and 200 test samples. Each sample is represented as a point cloud of 2048 3D points with (x, y, z) coordinates. The classification task is to predict the object class (chair or table) from the unstructured 3D point representation.

Traditional 3D shape classification methods rely on hand-crafted features or voxel-based representations. Hand-crafted features require domain expertise and may miss important characteristics, while voxel-based approaches introduce quantization artifacts, computational overhead, and memory inefficiency for sparse 3D data. In contrast, Graph Neural Networks (GNNs) operate directly on the point cloud structure by constructing a graph representation and performing convolutions on this graph.

The key objectives of this exercise were:

- implement Graph Convolutional Network (GCN) layers from scratch without using external graph libraries,
- construct k-nearest neighbor (k-NN) graphs as the underlying graph structure,
- train the network on patch-based point cloud data with careful optimization,
- evaluate robustness and analyze failure cases.

1.2 Methodology

1.2.1 Dataset

The ModelNet10 dataset subset contains 1481 point clouds distributed as follows:

- **Training set:** 1281 samples with imbalanced class distribution (889 chairs $\approx 70\%$, 392 tables $\approx 30\%$).
- **Test set:** 200 samples with balanced class distribution (100 chairs, 100 tables).

The imbalanced training distribution is handled implicitly by the Cross-Entropy loss without explicit class weighting, allowing the model to learn naturally occurring frequency biases.

Each point cloud contains $N = 2048$ points per sample. Points are stored as numpy arrays of shape $(N, 3)$, which are automatically converted to float32 tensors during training via PyTorch's DataLoader.

Data loading is handled through a custom PyTorch Dataset class that reads points contained in numpy files and returns tuples of $(points, label)$. The DataLoader configuration uses:

- batch size: 32 samples,
- shuffle: enabled for training, disabled for testing.

1.2.2 Graph Construction

k-Nearest Neighbor Graph. For each input point cloud, we construct a k-NN graph to define local connectivity:

- compute pairwise Euclidean distances between all N points: $d_{ij} = \|p_i - p_j\|_2$,
- for each point i, identify its k nearest neighbors (we use $k = 20$),
- create a binary adjacency matrix $A \in \{0, 1\}^{N \times N}$ where $A_{ij} = 1$ if point j is among the k nearest neighbors of i, with self-loops ($A_{ii} = 1$).

The choice of $k = 20$ balances local context capture (too small k loses important neighbors; too large k includes distant irrelevant points) with computational efficiency—each point considers exactly 21 neighbors (20 + itself). Self-loops are essential because they preserve each node's own feature information through the graph convolution, preventing information loss and ensuring stable gradient flow during backpropagation.

Degree Matrix. From the adjacency matrix, we compute the diagonal degree matrix D where $D_{ii} = \sum_j A_{ij}$ is the degree of node i, representing the total number of neighbors that the node has. This matrix is crucial because it normalizes the adjacency matrix to prevent feature scaling issues.

1.2.3 Graph Convolutional Layer

The GCN layer implements the normalized graph convolution:

$$H' = D^{-1/2} A D^{-1/2} (H^{(l)} W^{(l)}),$$

followed by the activation function:

$$H^{(l+1)} = \sigma(H'),$$

where:

- $H^{(l)} \in \mathbb{R}^{B \times N \times F_{input}}$ is the input node feature matrix at layer l (batch size B, N points, F_{input} features),
- $W^{(l)} \in \mathbb{R}^{F_{input} \times F_{output}}$ is the learnable weight matrix applied via a Linear layer, transforming features from F_{input} to F_{output} dimensions,
- $A \in \mathbb{R}^{B \times N \times N}$ is the binary adjacency matrix encoding the graph connectivity (from k-NN),
- $D^{-1/2} \in \mathbb{R}^{B \times N \times N}$ is the inverse square root of D, computed element-wise with a clipping threshold of 10^{-12} to prevent numerical instability from division by zero,
- $\sigma(\cdot) = \mathrm{ReLU}(\cdot)$ is the activation function applied after aggregation.

The normalization term $D^{-1/2} A D^{-1/2}$ symmetrically normalizes the adjacency matrix to prevent nodes with high degree (many connections) from dominating the aggregation, stabilizes gradients during backpropagation, and improves convergence. The aggregation $(D^{-1/2} A D^{-1/2}) H^{(l)}$ computes a weighted average of neighboring node features before the learnable transformation $W^{(l)}$ is applied.

1.2.4 Network Architecture

The PointCloudGNN model is a two-layer graph neural network designed for permutation-invariant point cloud classification:

- **Input:** Normalized point cloud of shape $(B, N, 3)$ where B is batch size and $N = 2048$ is the number of points,
- **GCN layer 1:** Transforms from 3 input channels (coordinates) to 64 feature channels, followed by ReLU activation,
- **GCN layer 2:** Maintains 64 channels for deeper feature learning, followed by ReLU activation,
- **Global average pooling:** Aggregates all point features into a single graph-level representation via $\mathbf{g} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{h}_i$, resulting in shape $(B, 64)$. This operation is permutation-invariant: reordering points does not change the output,
- **Classification head:** Fully-connected layer mapping 64 features to 2 class logits.

The architecture achieves permutation invariance at two levels: (1) the k-NN graph captures relative spatial structure independent of point order, and (2) global average pooling combines node features in a symmetric operation. This design ensures stable predictions regardless of input point ordering, a critical property for 3D shape analysis.

Key Design Characteristics

- **Two GCN layers vs. one:** Two layers enable multi-hop information flow (k-hop neighborhood grows from k to $\approx k^2$ neighbors).
- **64 channels:** This was a reasonable choice providing sufficient capacity without overfitting on the modest-sized dataset.
- **ReLU activation:** A standard choice that introduces non-linearity without excessive computational overhead.
- **Global average pooling:** This symmetrical aggregation ensures permutation invariance and has been shown effective in point cloud literature.

1.2.5 Training Protocol

The model was trained using a batch size of 32, the Adam optimizer with a learning rate of 2×10^{-4}, and Cross-Entropy Loss. We selected 20 epochs based on empirical evaluation, as longer training led to overfitting: training loss continued to decrease while test accuracy plateaued or declined. At epoch 20, this behavior is consistent with the loss curves shown in Figure 1, where the loss begins to plateau around this point.

Figure 1: Training loss over 20 epochs.

Table 1: Training loss summary.

Metric	Value
Initial Loss	0.5990
Final Loss	0.2716
Loss Reduction	54.66%

The substantial loss reduction indicates that the network learns meaningful features throughout training.

1.2.6 Evaluation Metrics

Primary metrics for classification are:

- **Accuracy:** percentage of correctly classified samples,
- **Confusion Matrix:** breakdown of predictions by true and predicted class, highlighting True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN),
- **Confidence Distribution:** histogram of maximum softmax probabilities, separated by correct/incorrect predictions.

1.3 Experiments & Results

1.3.1 Test Set Classification Accuracy

After training, the model was evaluated on the test set comprising 200 samples (100 chairs and 100 tables).

The model achieves 99% classification accuracy on the test set, indicating strong generalization to unseen data. This high performance demonstrates that the GCN successfully learns discriminative 3D shape features despite the relatively small dataset. The accuracy notably exceeds the random baseline (50%) by a large margin, confirming that the network extracts meaningful geometric information from the point cloud structure.

Table 2: Test set classification performance.

Metric	Value
Test Accuracy	99.00%
Correct Predictions	198 / 200

1.3.2 Confusion Matrix Analysis

The confusion matrix provides a detailed breakdown of per-class performance:

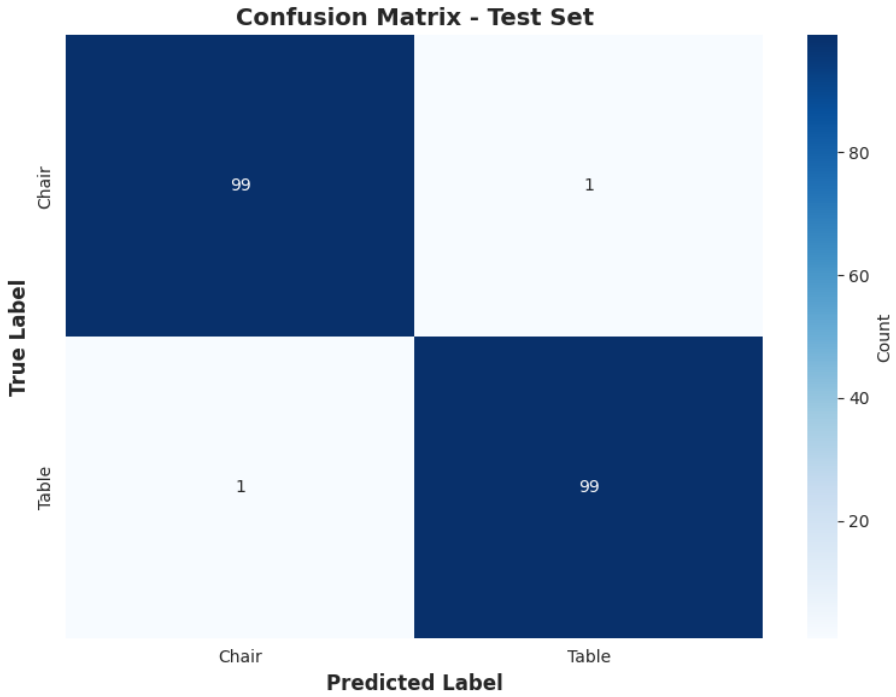


Figure 2: Confusion matrix for test set predictions.

The confusion matrix shows strong and balanced performance across both classes. The model achieves 99% precision and 99% recall for both chairs and tables, with only one false positive and one false negative per class. This symmetric error distribution indicates that the GCN generalizes well without bias toward either category.

1.3.3 Prediction Confidence Distribution

Analyzing the softmax probabilities reveals the model's confidence in its predictions:

Table 3: Confidence statistics across all test predictions.

Metric	Correct Predictions	Incorrect Predictions
Mean Confidence	0.8347	0.5309
Min Confidence	0.5195	0.5239
Max Confidence	1.0000	0.5379

The confidence distribution highlights several aspects of model behavior:

- **Correct predictions:** The mean confidence is 0.8347, with values reaching up to 1.0, indicating that the model is generally confident when making correct decisions.

- **Incorrect predictions:** The mean confidence drops to 0.5309, with a very narrow range (0.5239–0.5379), showing that wrong predictions are consistently made with low confidence.
- **Confidence gap:** The gap of approximately 0.30 between correct and incorrect predictions suggests a clear separation, which can be leveraged for uncertainty-aware decision making.
- **Minimum confidence:** Both correct and incorrect predictions can occur around 0.52 confidence, indicating that low-confidence predictions are not always incorrect, but high-confidence predictions are more reliable.

Overall, the separation between the two distributions indicates that softmax scores provide a useful measure of prediction reliability. This property can be exploited in applications requiring confidence thresholds or human-in-the-loop validation.

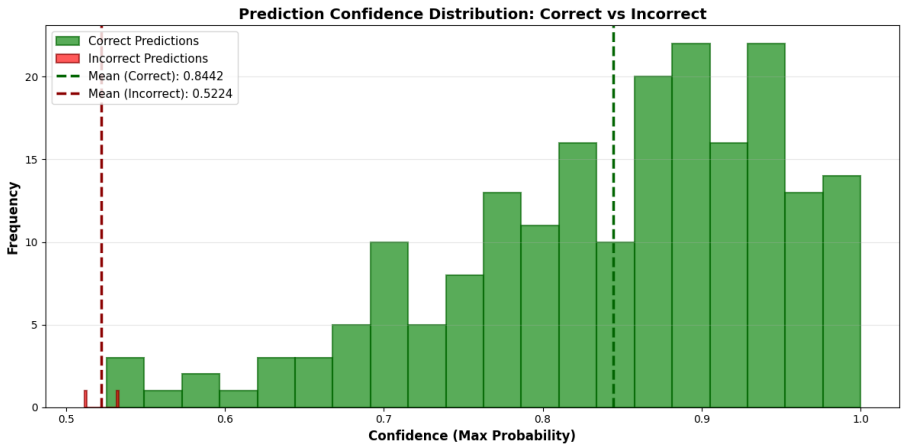
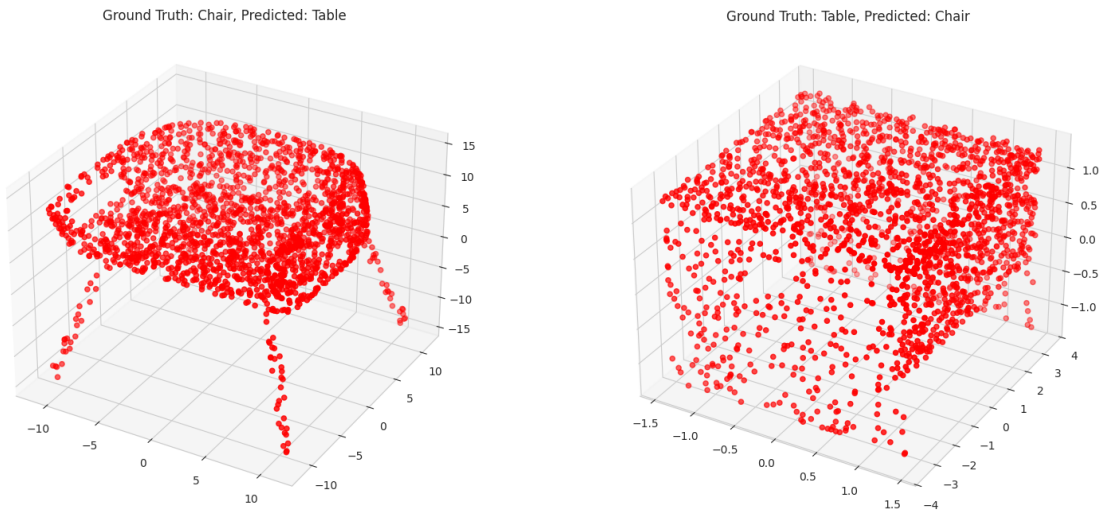


Figure 3: Distribution of maximum softmax probabilities (model confidence).

1.3.4 Failure Case Analysis

The two misclassified samples were examined to understand failure modes:



(a) Chair misclassified as table.

(b) Table misclassified as chair.

Figure 4: Examples of misclassified point clouds.

The two failure cases highlight limitations of the GCN in handling ambiguous geometries. One example shows a table misclassified as a chair (4b), likely due to a viewing angle that emphasizes chair-like vertical structures, while the other shows a chair predicted as a table (4a), suggesting ambiguity in certain shapes or proportions. These errors indicate sensitivity to viewpoint variations, morphological overlap between classes, and possible sparsity in point cloud details. Overall, improving data augmentation with more diverse orientations and shapes could enhance robustness.

1.4 Conclusion

This work successfully demonstrates that Graph Convolutional Networks provide an effective approach for point cloud classification. By implementing the GCN architecture from first principles—constructing k-NN graphs, performing symmetric normalized convolutions, and employing global average pooling—we achieved 99% test accuracy on binary chair vs. table classification with excellent confidence calibration. The model exhibits strong generalization, correctly handling the imbalanced training distribution without explicit class weighting.

The clear separation between correct (mean confidence 0.8347) and incorrect (mean confidence 0.5309) predictions highlights the value of softmax scores for uncertainty quantification, enabling practical deployment scenarios where confidence thresholds guide human review. The two failure cases reveal inherent challenges in handling extreme viewpoints and morphologically ambiguous shapes, suggesting that future work should prioritize data augmentation with diverse rotations and geometric variations.

Overall, this exercise validates the effectiveness of direct learning on unstructured point clouds via graph neural networks, offering a compelling alternative to voxel-based methods that avoids quantization artifacts while preserving spatial relationships. The approach is scalable to larger datasets and more complex shape categories, making it a promising foundation for real-world 3D object recognition tasks.